

MEMORY MANAGEMENT

The time when only one program was loaded simultaneously into the memory of a computer system is obviously long gone. Today, as we have already mentioned, the operating system has to manage many processes, as otherwise it would waste most of the processor capacity. One of the most important and complex tasks is therefore to have all these processes efficiently share memory, which has been scarce since the development of the first computer systems. Given the large number of processes, only some of them will be (partially) present in the main memory at any point in time. The operating system is thus responsible for determining on the one hand which processes are present in the memory (something we will come back to later) and on the other hand how the processes present (i.e., their images) are logically and physically organized, which is the subject of this chapter. We already know that secondary, or still virtual, memory is used, typically a portion of disk space. So processes, after they have been started and loaded into memory, may have to make room for other processes several times. When, by a scheduling decision of the operating system, they are loaded back into main memory, it does not necessarily have to be in the same location. In addition, the memory space of the processes must also be protected against unauthorized access by other processes and the possibility must exist to share areas of memory.

Loading and linking

A first item closely related to memory management is how the object code (i.e., the instructions plus data section) of a process is loaded into memory. There are a number of possibilities for this. First, the object code may use *physical/absolute* addresses, determined by the compiler (or programmer). In this case, the *loader*, which loads the object code into memory, has little choice: the code can only be read in one place, since the addresses in the *branch* instructions and data references would otherwise be incorrect. The loader has a very simple task in this case, of course.

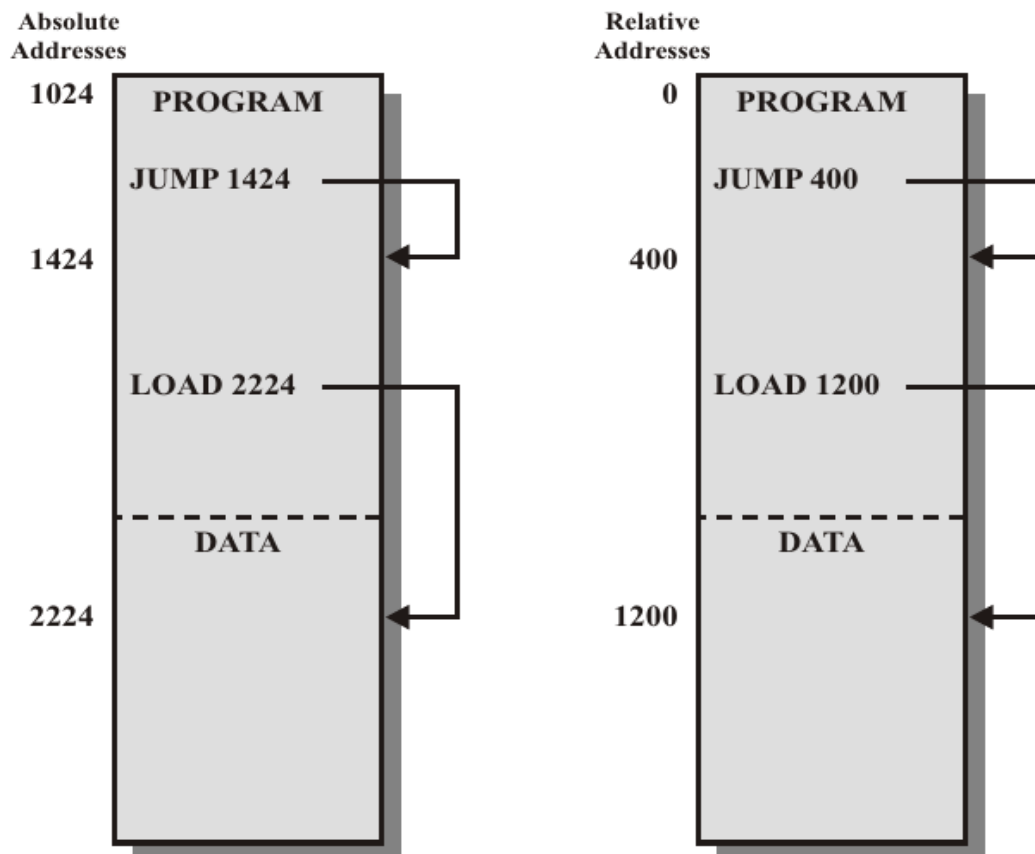


Fig. 16: Absolute and Relocatable load modules [Stallings 03].

A slightly more flexible possibility is that the object code is created by the compiler with *logical/relative* addresses. In this case, each address will be relative to some place in memory, usually the starting point (see Figure 16). These logical addresses still need to be translated into physical addresses. When this is done when loading the object code, it is called *relocatable loading*. The load module will replace each logical address with a physical address since it knows where the code will be stored. Information indicating where the addresses to translate are located is created by the compiler (i.e., the *relocation directory*). Relocatable loaders have the advantage that the location of a process in memory is not fixed in advance. However, once loaded, the physical addresses are fixed and the process must be reloaded to the same location each time it was referenced to swap space for a while

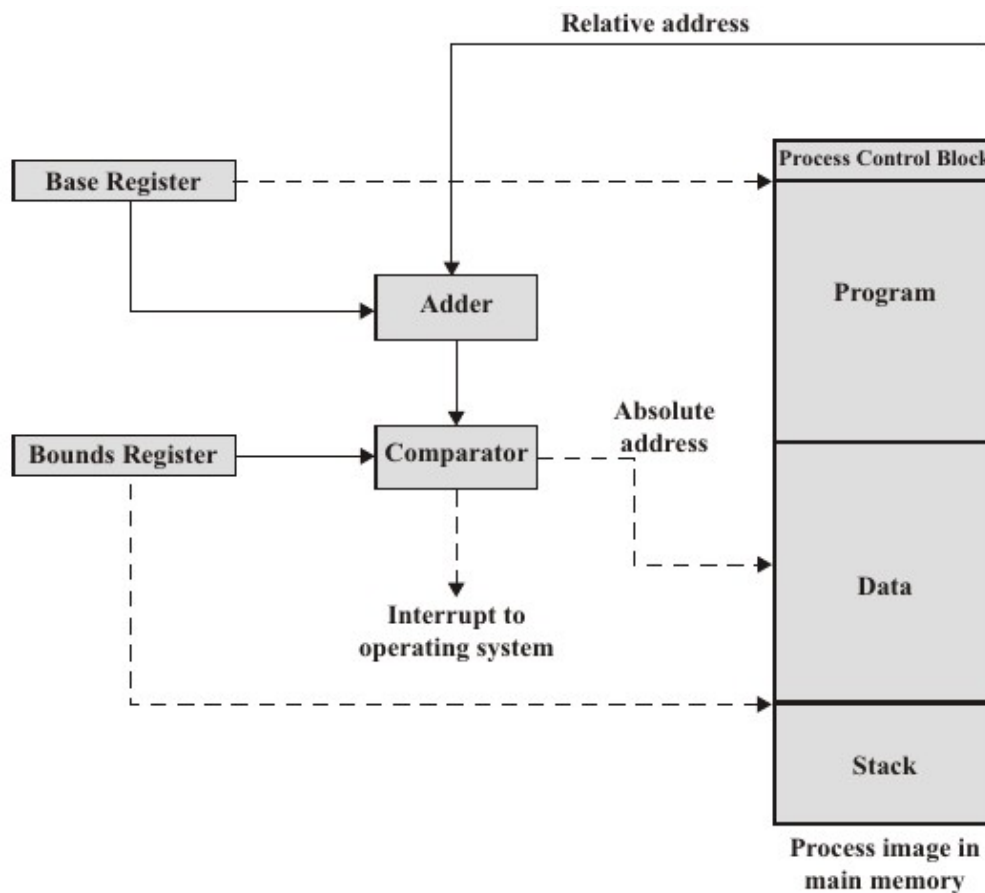


Fig. 17: Hardware support for address translation [Stallings 03].

This gives rise to a very inefficient use of memory.

The solution is to perform the address translation only at *run-time*, i.e. when the instruction is actually executed. The loader will then load the object code with logical addresses and the processor itself will take care of the translation using specially designed hardware. This has the great advantage that the same process, during its lifetime, can now be loaded without problems into different memory locations. The operating system has the possibility to perform *relocation*. A graphical representation of the hardware support for relocation is shown in Figure 17, where for simplicity we assume that the process image is continuously present in memory. The code of a process will of course in many cases not consist of a single object code file, but of multiple such files (which simplifies the reuse of code considerably). In that case the different object code files must also be linked to each other by the so-called *linker module*, so that references to the code and data of the other modules are also done correctly. This can be done during compilation or when loading the module. Finally we also note that the process control block and the process stack together with the loaded object code form the process image.

1 Partitioning

Now that we have these considerations behind us, we can turn to the main focus of this chapter: memory organization. There have historically been three main ways to organize the memory of a computer system: *partitioning*, *paging*, and *segmentation*. Since the latter two greatly outperform the former in terms of performance, partitioning is still found only in the kernel portion of some operating systems. However, we will also take a closer look at this method of organization since it raises some important issues that are avoided by the latter two techniques. Partitioning is also the simplest and oldest of these techniques.

In most systems, part of the memory will be reserved for the operating system. Our attention is mainly focused on the management of the remaining memory to be shared by the different user processes. In addition, secondary, or virtual memory will very often be used, a topic to which we will return later in the course. For now, we will limit ourselves to the main memory

1.1 Fixed partitioning

A first option is to divide the memory into a number of parts, called partitions. These can be of fixed or variable size. A process that needs to load into memory will have one of these partitions at its disposal. When all partitions are the same size the loading of processes is simple: if a partition is free we use it for the process to load. What we notice here is that each process gets an equal share of memory, no matter how much memory it requires. This gives rise to what is called *internal fragmentation*: even when a partition is occupied by a process, on average, half of the partition remains unused. In addition, it can also happen that a process is too large to fit into a partition.

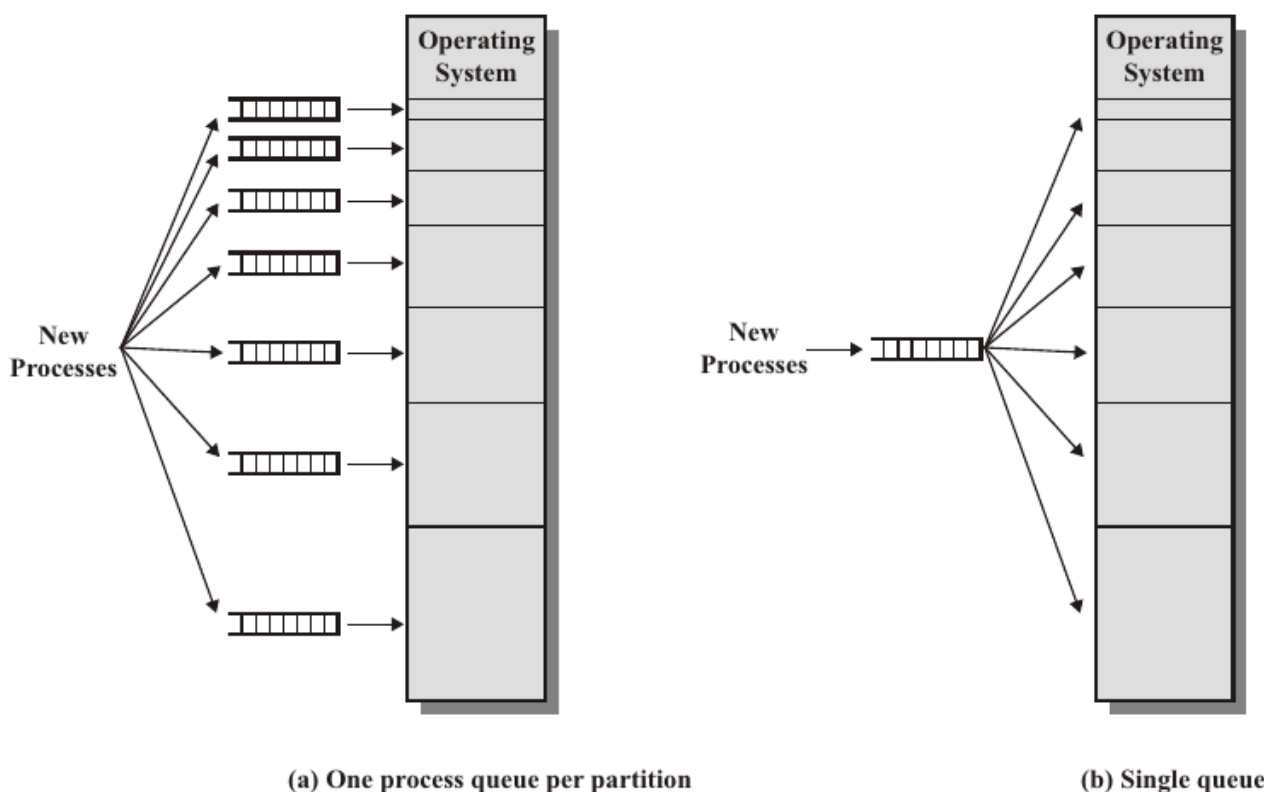


Fig. 18: Fixed partitions with variable size: process placement [Stallings03].

Partitions of variable length partially solve both of these problems. By offering a variation of partition sizes, less memory is lost and we can support larger processes. However, it remains very difficult to determine in advance how best to choose the partitions. An additional question is how to select a partition when we want to load a new process into memory. To avoid internal fragmentation it is best to select the smallest possible partition size that surrounds the process

image. Possibly all partitions of this size are already in use. We then have two options (see Figure 18): (1) either let the process wait until one of these partitions becomes free, (2) we can also opt for another, free partition that is sufficiently large. The first choice maximizes memory usage within a partition, while option two may give rise to better usage in terms of the number of partitions.

1.2 Dynamic partitioning

Since fixed partitions give rise to *internal fragmentation*, the solution seems simple: we create the partitions dynamically so that they are exactly the required size. However, it should be taken into account that new processes are added and removed all the time. An example can be seen in Figure 19, note that process 2 in step (h) can also be a new process (process 5).

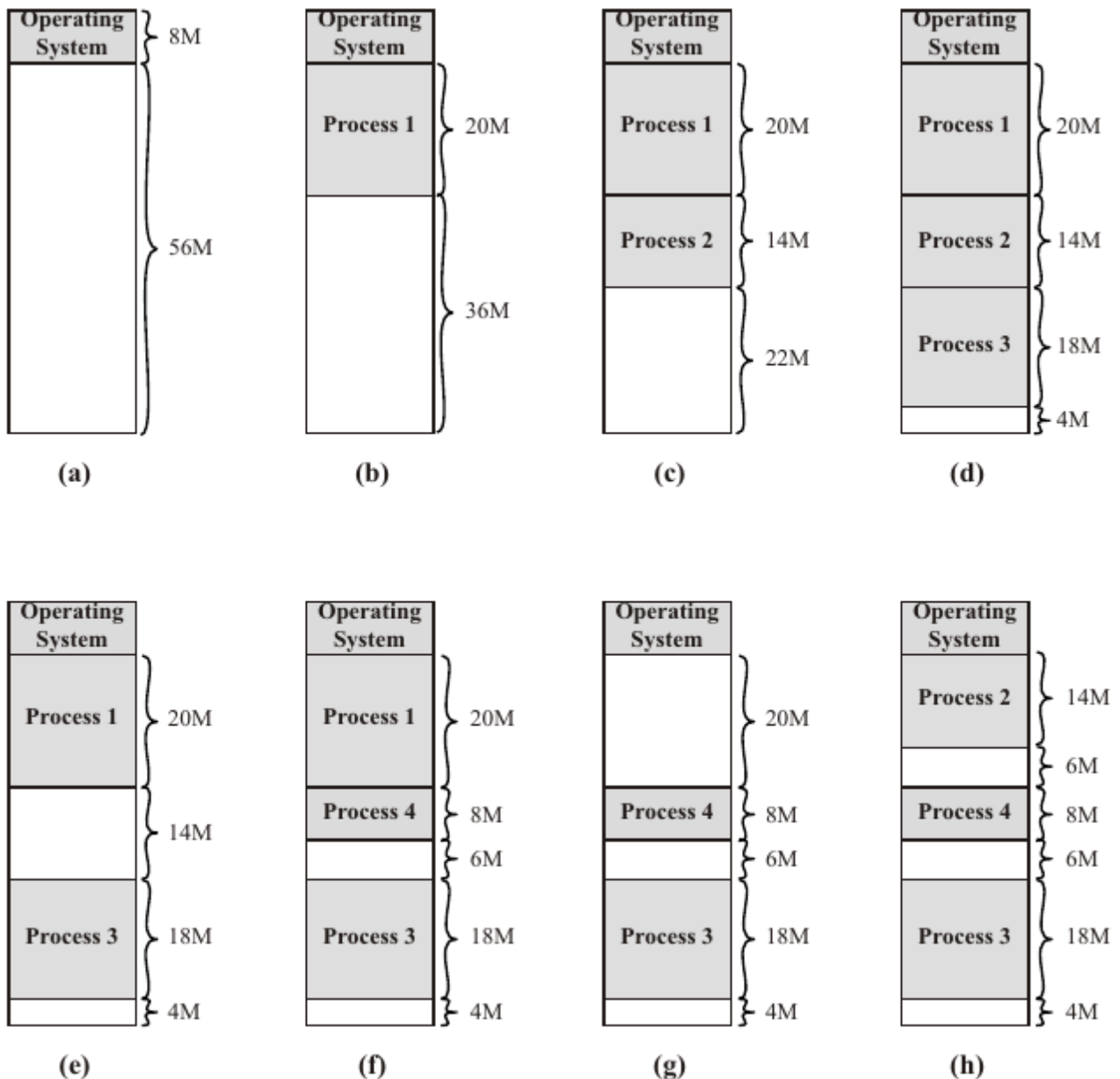


Fig. 19: Evolution of memory under dynamic partitioning [Stallings03].

We observe that unused areas of memory arise between partitions, which we call holes (or voids). The creation of these voids, is called *external fragmentation*. When a new process emerges, we must also determine where we will place the new partition. Here we can try to fill the voids that arise. Once we have determined in which void we will place the partition, we make sure that it connects to the previous partition so that we do not create an additional void.

How many voids can we expect in such a system? Suppose we observe memory for a long time and note p_i as the probability that a process image is surrounded by i voids, for $i = 0, 1$ and 2 . Then, on average, the number of voids will increase by $p_0 - p_2$ when a process leaves memory. Namely, if there are two voids, they will be merged, reducing the number by one. If there are no voids, then one is added. When adding, with the exception that a partition fits exactly, the number will remain unchanged. The average number of voids will not change over time, so p_0 must equal p_2 . The average number of voids around a process is exactly equal to $p_1 + 2p_2 = p_1 + p_2 + p_0 = 1$, by using the derived equality. In short, on average there are twice as many processes present as voids (since a void is always surrounded by two processes). This result is known as the 50 percent rule.

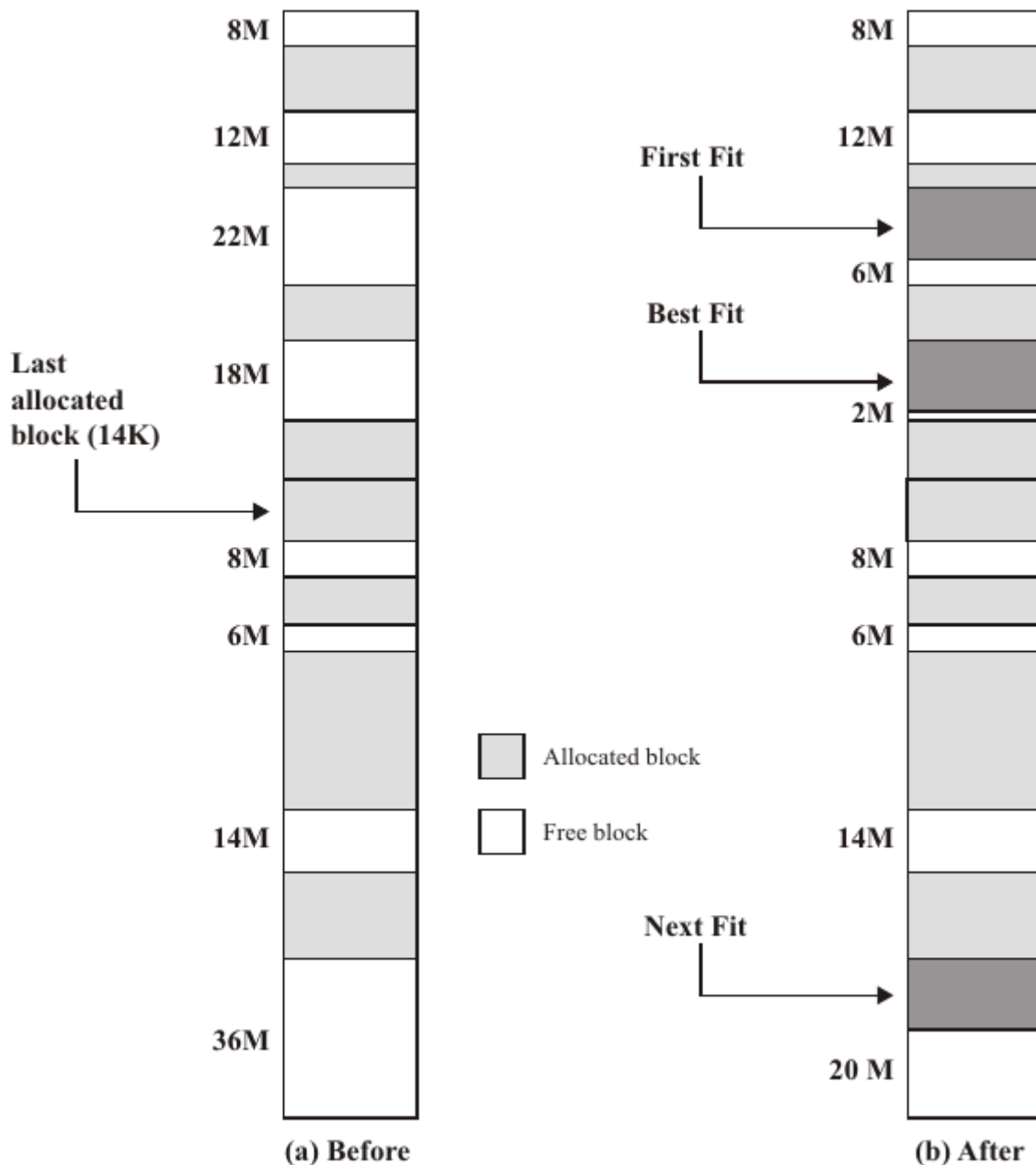


Fig. 20: Placement algorithms under dynamic partitioning [Stallings 03].

The placement of partitions can be done using a number of algorithms (see also Figure 20). Thus, we can look for the smallest possible void that is sufficiently large (Best-fit), we can locate the first void that is sufficient (First-fit), the next one relative to the last allocated partition (Next-fit) or even the largest void (Worst-fit). At first glance, the Best-fit algorithm seems the most efficient, since it tries to fill the gaps as best as possible. However, Best-fit creates very many rather small voids, which may be of little use for loading other processes and as such this method is not always the

most efficient. For example, assume we have two voids of size 20 and 70 Kbyte and all processes require at least 12 Kbyte. If the next process demands 12 Kbyte, then Best-fit will create a void of 8 Kbyte, which is unusable by all the other processes. In this case, it is better to wait until a process emerges that better fills the 20 Kbyte. Best-fit creates this type of voids frequently.

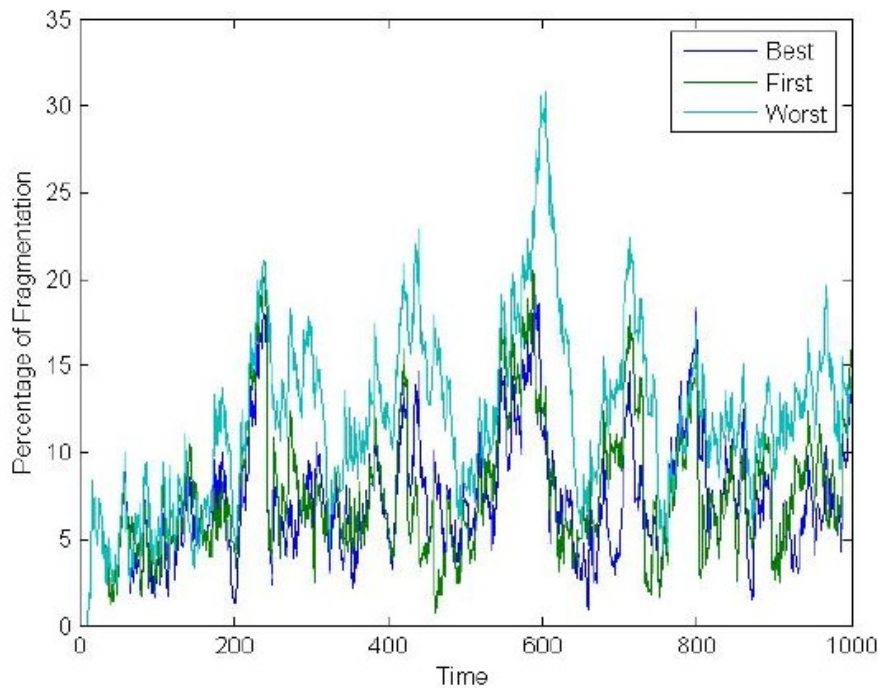


Fig. 21: Measured fragmentation for Uniformly distributed process lengths between 1 and 40 units.

The other extreme is the Worst-fit, it will choose the largest void precisely to counteract the above phenomenon. However, by always opting for the largest void, large processes will often fail to find a suitable void, requiring a large additional amount of memory to be allocated after it. So it should come as no surprise that the First-fit and Next-fit strategies are regularly the most efficient (although everything depends very much on the scenario). First-fit will mainly create many small voids 16 and 32 units up front (where the probability always halves when doubling the length). Next-fit is usually faster in execution because, unlike First-fit, it does not have to keep going over the series of small voids located in the front of the memory. Consequently, the Next-fit strategy has been used extensively in the past.

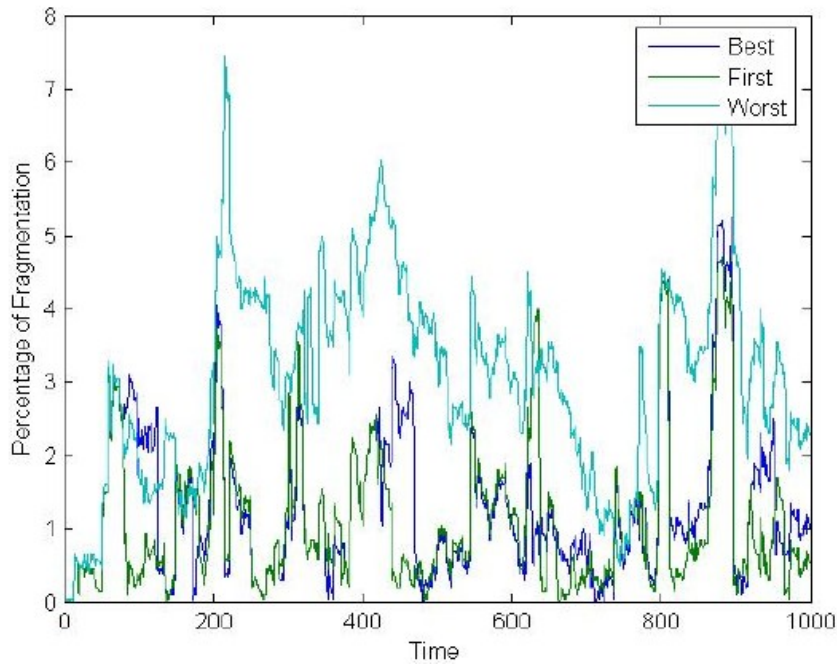


Fig. 22: Measured fragmentation for Spiky distributed process lengths 1, 2, 4, 8,

Experimental confirmation of the above considerations for Best-fit, First-fit and Worst-fit is shown in Figures 21 and 22. In this setup, a new process serves at each time point, the processes all have durations of geometric duration with an average of 20 time points. Sufficient memory is provided so that no process can ever fail to join the memory. The measured fragmentation corresponds to the sum of all voids between processes, including the possible void at the front of the memory (but of course without the large void at the back).

One can also deal with the prevention of voids by *compaction*: periodically, the operating system will relocalize all processes so that their partitions form a contiguous block again. This operation is obviously very expensive in terms of processor time.

1 Mbyte block	1 M					
Request 100 K	A = 128 K	128 K	256 K	512 K		
Request 240 K	A = 128 K	128 K	B = 256 K	512 K		
Request 64 K	A = 128 K	C = 64 K	64 K	B = 256 K	512 K	
Request 256 K	A = 128 K	C = 64 K	64 K	B = 256 K	D = 256 K	256 K
Release B	A = 128 K	C = 64 K	64 K	256 K	D = 256 K	256 K
Release A	128 K	C = 64 K	64 K	256 K	D = 256 K	256 K
Request 75 K	E = 128 K	C = 64 K	64 K	256 K	D = 256 K	256 K
Release C	E = 128 K	128 K	256 K	D = 256 K	256 K	
Release E	512 K			D = 256 K	256 K	
Release D	1 M					

Fig. 23: Example of the evolution of the Buddy system [Stallings 03].

1.3 Buddy System

The *buddy* system is an interesting variant of fixed and dynamic partitioning. The idea is that the memory size is equal to a power of 2, suppose 2^H . The size of each partition is also a power of 2 located between 2^L and 2^H . For each of these $H - L + 1$ block sizes, the system will maintain a list of the (free) blocks with this size. Initially, there is only one block, of size 2^H , and thus all other lists are also empty. When a process requires a block of size k , with $2^{i-1} < k \leq 2^i$, the following recursive procedure is used:

```
procedure get hole(i);
begin
  if i_list empty then begin
    get hole(i+1);
    split hole into buddies;
    put buddies on i_list;
  end;
  take first hole on i_list
end;
```

If a block of the correct size is not present, a larger free block will be divided (recursively) until a block of the correct size is created (see also Figure 23). The state of the buddy system can be easily displayed by means of a tree structure. If a block is released again, because a process is removed from memory, then this block may be rejoined with its *buddy* to a larger block, at least if the buddy itself is also (completely) free. So in the tree representation there is always one of both children (partly) filled. A variant of the buddy system is used in the kernel memory allocation of certain UNIX systems.

We can also easily estimate how much extra memory a process image is allocated, assuming that the size of the process image has a random length smaller than the available memory. Namely, if the image is b bytes long, then a chunk of $2^{\lceil \log_2 b \rceil}$ bytes will be allocated (note, $2^{\lceil \log_2 b \rceil}$ is the least power of 2 that is greater than b). The first half of this piece is certainly utilized, while on average we use 50% of the second half. In short, $3/4$ of the allocated part is effectively used by the image. In other words, an image is allocated 33% more memory than necessary (i.e., $4/3 \approx 1.33$).

2 Paging

An alternative method for partitioning consists of dividing the entire memory into small blocks of fixed size, which we call *page frames* or simply *frames*. The image of a process is also divided into a number of pieces, which we call *pages*. These pages have the same size as the frames into which the memory is divided. When we now wish to load the process into memory there will be exactly enough frames allocated. An important difference with partitioning is that these frames do not have to form a *contiguous* block in memory. This technique is called *paging*.

So with paging we have no *external* fragmentation, there is still a very limited form of internal fragmentation as the size of the process image does not necessarily have to be a multiple of the frame/page size. On average, however, it is only about half of a page per process, unlike partitioning.

Paging, of course, requires additional work in translating the logical to physical addresses. After all, an address now consists of a page number and an offset value within this page. The processor can no longer simply add the logical address to the base register value, since the pages are scattered throughout the memory. To determine the correct address will therefore, per process, a *page table* be maintained. This table contains the numbers of all pages of a process (at the i th place we find

the page frame number of page i , which allows for quick lookup). The table is thus always consulted Process segment when table we need to determine a physical address.

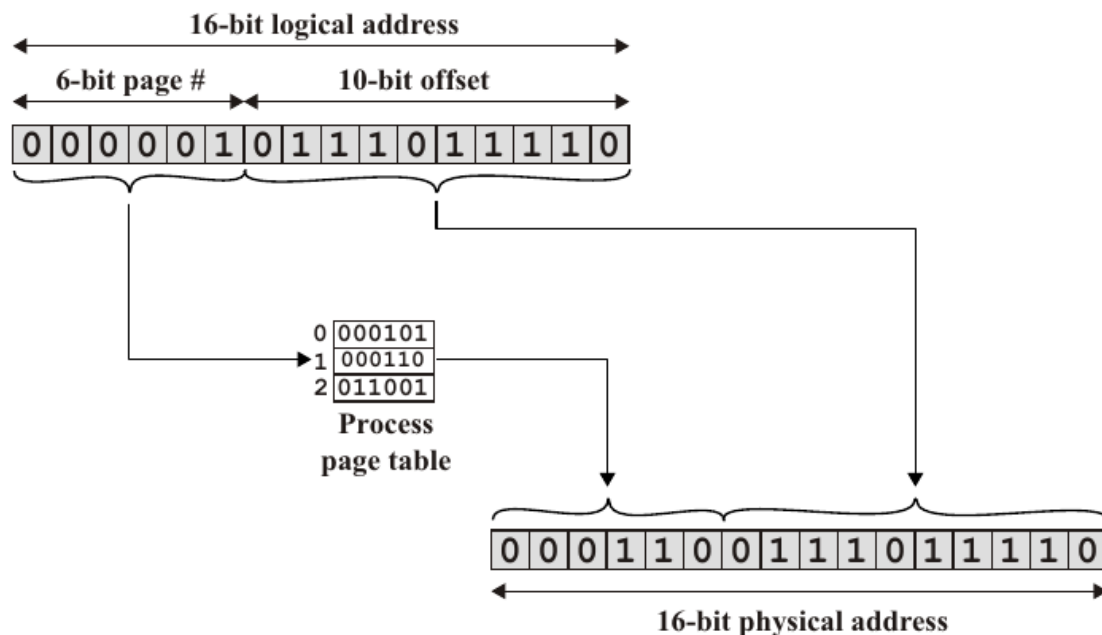


Fig. 24: Address translation for paging [Stallings 03].

Suppose a logical address consists of $n + m$ bits and 2^m is the size of a page frame in bytes. Then the first n bits will be the page number, while the remaining m bits indicate the offset within the page. To translate a logical address it suffices to take the first 16-bit n physical bits of address the logical address and use it as an index to the page table to get the frame number. This frame number constitutes the first n bits of the physical address. Then one simply adds the m last bits of the logical address (see Figure 24) to obtain the physical address.

The translation of the logical to physical addresses is done entirely by hardware. The hardware will take the first n bits of the logical address, consult the page table (which is stored in RAM memory) and then, when a frame number is obtained, stick the offset to it. Because this is done entirely in hardware, it is also the hardware that determines the format of a page table entry.



Fig. 25: Page table entry on 32 bit x86 architecture

On 32 bit machines using the x86 architecture, a page is 4096 bytes in size and a page number consists of 20 bits, allowing up to 4 Gbyte of memory to be addressed. A page table entry in this case, as shown in Figure 25, consists of 4 bytes and contains not only the 20 bit frame number, but also some additional bits. The P bit indicates whether the page is present in memory, if not then all remaining 31 bits can be used freely. The U/S bit indicates whether a page may or may not be accessed in user mode, which ensures that the memory used by the kernel cannot be accessed by user code. Then there is the R/W bit that indicates whether data may be written to a page. This is useful for ensuring that instructions from a process cannot be accidentally overwritten by errors in the user code. The D bit indicates if a page is *dirty*, which means that it has been modified and the A bit indicates if the page has already been accessed. Note that the page table also ensures that a process cannot write into the memory space of another process. Since the page numbers consist of 20 bits, the entire page table contains 2^{20} entries. This means that a page table would require

4MBytes of memory per process, which is very large. In the Virtual Memory chapter, we will show how to reduce the amount of memory required per page table.

When using paging, it appears to the programmer/compiler that the process image is present in memory as a coherent block. Hence, paging is said to be transparent to the compiler/programmer.

3 Segmentation

Paging can be interpreted as a fixed partitioning scheme with fixed partition sizes, with the difference that a process can now have multiple partitions that do not have to be located next to each other. As a result, only internal fragmentation occurs. Segmentation relates to the same way to dynamic partitioning as paging to fixed partitioning. Namely, a process may have one or more segments, which may be variable in size and are not necessarily present in memory as a contiguous block. Internal fragmentation is avoided by this approach, however, external fragmentation may occur. Compared to dynamic partitions, this will be less pronounced, given that a process image can be spread across multiple segments.

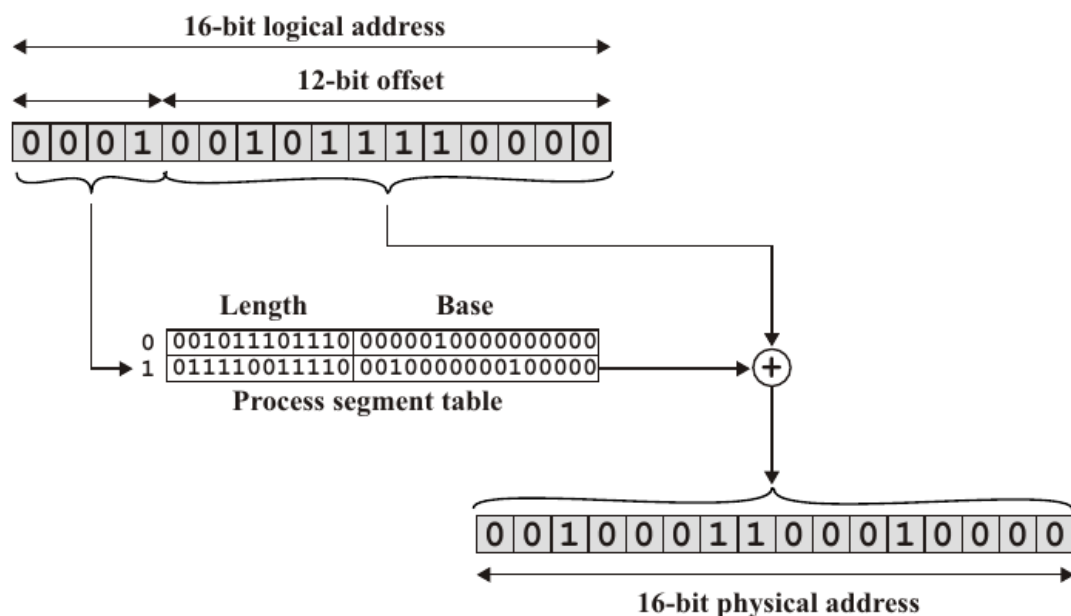


Fig. 26: Address translation for segmentation [Stallings 03]

A logical address of $n + m$ bits consists, in the case of segmentation, of a segment part that includes the n left most bits and an offset formed by the remaining m bits. To convert this logical address into a physical address, a segment table is used. This table contains per segment the starting address of the segment, called the *base* value, and the length. This length is stored to the validity of an address to check. If it turns out that the offset formed by the m rightmost bits is smaller than the segment length, the offset is added to the base value to arrive at the corresponding physical address (see Figure 26).

Segmentation was initially introduced on the x86 architecture to ensure that more than 64 Kbyte of memory could be supported on a 16 bit machine (up to 1 Mbyte). On the x86 architecture, a logical address is first converted to a linear address via segmentation and then the physical address is obtained via paging (see Figure 27). However, the conversion of the logical to linear address via segmentation proceeds slightly differently than previously indicated. Thus use is made of 4 segment registers, each with 16 bits. These 16 bits determine (indirectly) the segment that is used as further indicated. The choice of the segment register used in determining the linear address is implicitly determined by the instruction. Thus, there is a register for the stack, for program code and two registers for data.

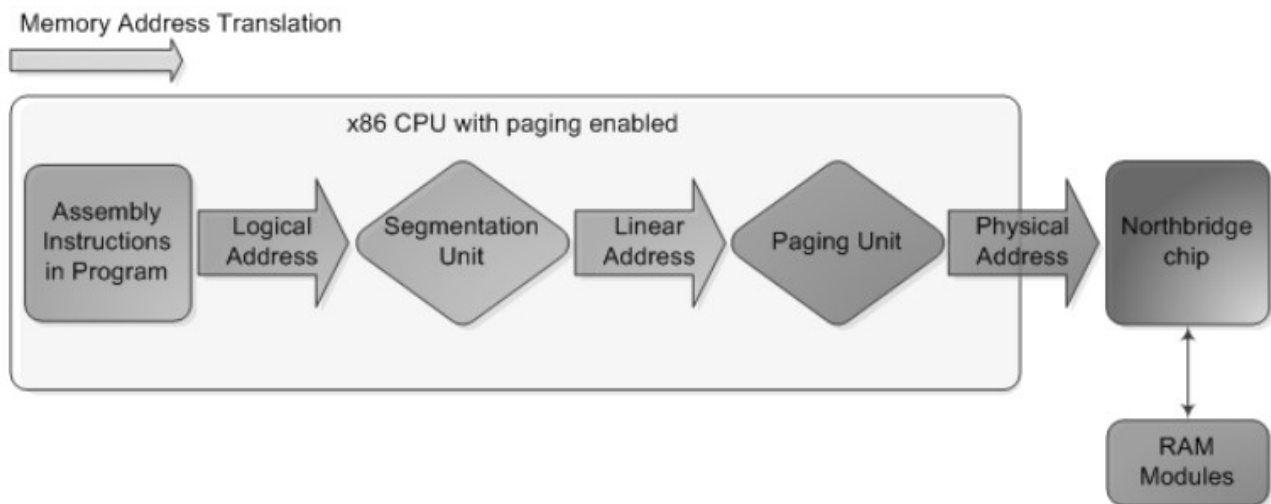


Fig. 27: Address translation on x86 architecture.

To convert the logical address to a physical address on the x86 there are two possible modes: *real mode* and *protected mode*. In real mode, the value in the segment register is multiplied by 16 and then the logical address is added to this to obtain the 20 bit linear address. Note that in this case the same linear address can be obtained by different combinations of a logical address and a segment number (something MS-DOS programmers often used, so they did not always have to adjust the value in the segment registers). Furthermore, this way one can also form addresses that are just slightly larger than 1 Mbyte. Since, on a machine with 1 Mbyte of memory, only 20 bits can be passed to RAM memory (via the Northbridge chip), the 21th bit was implicitly set equal to zero.

The programmer can (with the exception of the segment register for program code) also explicitly fill in the value of a segment register (via the MOV instruction) and in this respect segmentation is not transparent to the programmer. In protected mode, however, a segment table is used and the content of the used segment register is used as an index to the segment table. The segment table also provides protection by firstly working with a limit on the segment size (as indicated earlier) and secondly by keeping track of the privilege level required to use this segment. On the x86 there are 4 privilege levels but most operating systems use only two levels: level 0 (kernel mode) and level 3 (user mode). Real mode segmentation is used when the machine boots up and then switches to protected mode segmentation (after all, the segment table must be set up first). Although the x86 architecture still works with segmentation, since the introduction of 32 bit machines, the value of the segment registers in protected mode is often set equal to 0 (and the segment limit to 4 Gbyte), making the logical and linear addresses identical. In this case, we say that segmentation works in *flat mode*, which is always the case on 64 bit machines.